六、高阶 Agent

在前面的课程中，我们已经学习了 Agent 的基础概念、推理模式、Multi-Agent 协作（CrewAI）等内容。这些内容帮助我们掌握了"如何构建一个能工作的 Agent"。

但在真实的生产环境中，Agent 往往需要处理**更复杂、更动态、更持久**的任务。这时，我们需要进入 **高阶 Agent** 阶段。

高阶 Agent 的核心特征是：

- 不再是简单的工具调用或线性协作
- 能够与外部知识、工具、系统进行深度融合
- 具备更强的动态决策、长期记忆和安全控制能力
- 最终目标是实现**可靠、可扩展、可落地**的智能系统

本章我们将聚焦当前高阶 Agent 领域**最重要、最实用**的三个方向：

- **MCP 协议**
- **Skill**
- **Langgraph**

通过这三部分内容，你将掌握高阶 Agent 的核心能力。

1.MCP协议

1.1 概述

1.1.1 问题导入

在前面的学习中，我们已经掌握了如何让多个 Agent 协作完成任务。然而，一个关键问题仍悬而未决：

Agent 如何高效、安全地连接外部工具和数据?

回顾我们曾讨论过的问题——Agent 系统中**工具接入的碎片化**正日益凸显。现代 AI 系统平均需要集成 7 至 12 种外部工具，涵盖数据检索、知识图谱、计算引擎等不同类型。当我们需要让 Agent 同时使用 GitHub、PostgreSQL、Slack 时，问题迅速升级。

想象这样一个场景：你有 10 个 AI 应用和 100 个工具，理论上需要 1,000 个不同的自定义连接器。这不仅意味着巨大的工程负担，更关键的是——**每次切换 Agent 框架或更换 LLM 供应商，都需要重写整套工具适配代码。**从 Multi-Agent 系统的视角来看，这是一个典型的 **N×M 倍增问题**：N 个 Agent × M 个工具 = N×M 个定制集成点。

面对这一挑战，业界涌现出两种主流解决路径：

维度	MCP 标准化协议	Agent + Function Call 模式
架构	Client/Server 解耦	工具逻辑嵌入 Agent
集成方式	一次开发，MCP 客户端通用	为每个 LLM 平台单独适配
工具管理	Server 统一封装与维护	分散在各 Agent 中，难以复用

维度	MCP 标准化协议	Agent + Function Call 模式
安全边界	隔离运行，权限独立	与 Agent 进程共享，风险耦合
适用场景	企业级、多工具复杂系统	快速原型、推理主导的任务

MCP 采用 Client-Server 架构，强调工具与模型解耦，适合企业级应用；而 Agent + Function Call 则更灵活，适合快速开发和推理驱动的任务，但安全性和可控性较弱。

1.1.2 MCP发展历程

MCP 自推出以来，发展速度极快，已迅速成为高阶 Agent 领域的重要开放标准之一：

时间节点	里程碑事件
2024 年 11 月	Anthropic 开源 MCP，最初主要服务于 Claude 与外部工具、数据源的连接
2025 年	OpenAI、Google、Microsoft 等主流平台和开发生态开始陆续兼容或支持 MCP 相关能力
2026 年	MCP Server 生态持续扩展，覆盖文件系统、数据库、代码仓库、浏览器、企业 SaaS 等多类工具
2026 年 Q1	越来越多企业开始将 MCP 作为 Agent 工具接入层的标准化方案之一

截至目前：

- MCP 已经成为 Agent 工具接入和上下文连接领域的重要开放协议。
- 越来越多的开发者和企业团队开始使用 MCP Server 统一封装内部工具、数据库和业务系统。
- 在复杂 Agent 系统中，MCP 常被用作“工具标准化接入层”，与 LangGraph、CrewAI、AutoGen 等 Agent 框架互补使用。

这一发展速度充分说明：MCP 正在从“新兴协议”快速成为高阶 Agent 生产落地中的关键基础设施之一。

1.1.3 MCP 的定位与 Agent 框架的关系

MCP 并非要替代现有的 Agent 框架，而是作为**标准化基础设施**，增强 Agent 的“四肢”能力（即工具调用与外部交互能力）。

它与 CrewAI、LangGraph、AutoGen 等框架形成**互补关系**：

- **MCP** 负责标准化暴露工具和数据能力
- **Agent 框架** 负责编排多 Agent 的协作流程、交互模式和决策逻辑

前文介绍过的 Agent 交互模式（协作型、竞争型、混合动机），可以通过 MCP 提供的标准化通信协议来实现实际的工具调用。**两者共同构成完整的 Multi-Agent 系统设计。**

1.1.4 实际应用场景

MCP 协议已在企业级 Agent 系统中得到越来越多的应用，主要解决“工具碎片化”和“跨系统集成”两大痛点。典型应用场景包括：

- **企业内部工具集成**：将 ERP、CRM、OA、数据库等系统工具统一接入 Agent，实现“一键调用”。
- **跨平台协作**：不同框架（CrewAI、LangGraph、AutoGen）的 Agent 可以通过同一 MCP Server 调用相同工具，无需重复开发。
- **生产环境工具管理**：支持敏感操作的权限控制、审计日志和隔离执行，满足企业安全合规要求。
- **动态工具生态**：企业可以搭建统一的 MCP Server 平台，让内部多个 Agent 项目共享工具能力。

1.1.5 MCP 的核心优势

与传统 Tool Calling 相比，MCP 具有以下显著优势：

优势维度	传统 Tool Calling	MCP 协议	实际价值
标准化程度	碎片化，不同框架格式不同	统一工具描述与调用协议	大幅降低适配成本
工具发现能力	静态配置，无法动态发现	支持运行时动态发现	工具可即插即用
安全性与可控性	权限控制弱，难以审计	可结合权限控制、隔离运行与审计机制	满足企业级安全合规要求
生态复用性	工具仅限单个框架	一次实现，多客户端复用	构建企业级工具生态
开发维护成本	高（重复开发适配代码）	低（标准化 + 动态发现）	显著降低长期维护成本

核心价值总结：

MCP 让 Agent 从“各自为战、工具重复开发”的阶段，进入“标准化、可插拔、生态化”的新阶段，是高阶 Agent 走向生产落地的关键基础设施。

1.1.6 依赖安装

MCP 是协议，`mcp` 是实现这个协议的 Python SDK（软件开发工具包）。

- **MCP**：是一套标准、规则、Schema 定义（类似于 HTTP 协议、OAuth 协议）。
- **mcp**（小写）：是基于这个协议开发的 **Python 实现包**（Python SDK）。

就像：

- HTTP 是协议 → `requests` 是使用 HTTP 协议进行网络请求的 Python 包
- MCP 是协议 → `mcp` 是实现 MCP 协议能力的 Python 包

开发者在使用 MCP 时，通常会安装以下包：

```
# 最常见的安装方式
pip install mcp
```

安装后，就可以这样导入使用：

```
from mcp.server.fastmcp import FastMCP
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
```

1.1.6 MCP 协议的核心目标

MCP 协议的核心目标可以概括为三点：

- **标准化**：提供统一的工具描述、调用和返回格式
- **可发现性**：Agent 可以自动发现并加载可用的工具
- **安全性与可控性**：支持与认证、权限控制、隔离运行和审计机制结合使用

通过 MCP，开发者只需一次实现工具服务，即可在多个 Agent 框架或客户端中复用，大幅降低开发和维护成本。

1.2 MCP 的核心架构

MCP 协议采用 **Client-Server 架构**，其设计核心是**标准化 + 可扩展 + 安全可控**。下面我们按照逻辑顺序，先介绍最基础的 **MCP Schema**，再逐步介绍其他组件。

1.2.1 MCP Schema（标准协议规范）

（1）定义

MCP Schema 是整个协议的**灵魂和基础**。它定义了工具描述、调用请求、返回结果以及错误处理的**统一标准格式**，确保不同框架、不同语言的 Agent 和工具之间能够无缝交互。

（2）**Schema 的核心作用**

- 让工具具有**自描述性**（Agent 可以自动理解工具能做什么）
- 统一参数类型、返回结构和错误格式
- 支持工具的动态发现和验证

（3）**示例 Schema 片段**（JSON Schema 风格）：

```
{
  "name": "get_stock_price",
  "description": "获取某只股票的实时价格",
  "inputSchema": {
    "type": "object",
    "properties": {
      "symbol": {
        "type": "string",
```

```
            "description": "股票代码，例如 AAPL"
        }
    },
    "required": ["symbol"]
  }
}
```

在实际 Python SDK 开发中，开发者通常不需要手写完整 JSON Schema。SDK 会根据函数签名、类型注解和 docstring 自动生成对应的工具描述。

教学要点：MCP Server 会以标准化 Schema 暴露工具，Agent 才能自动发现和调用。

1.2.2 MCP Server（工具服务端）

（1）定义

MCP Server 是工具的实际提供者，负责将各种外部能力（API、数据库、文件系统等）以 MCP 标准格式对外暴露。

（2）**主要职责**：

- 按照 MCP 规范注册工具
- 接收 Agent 的调用请求
- 执行具体工具逻辑
- 返回标准化结果
- 在生产环境中结合权限检查、日志记录和审计机制

（3）**代码示例**（可运行 MCP Server）：

```python
from mcp.server.fastmcp import FastMCP

# 创建 MCP Server
mcp = FastMCP("finance_tools")


@mcp.tool()
def get_stock_price(symbol: str) -> dict:
    """
    获取股票实时价格。

    Args:
        symbol: 股票代码，例如 AAPL、MSFT、TSLA

    Returns:
        包含股票代码、价格和时间戳的字典
    """
    # 教学示例：这里先返回模拟数据
    # 真实项目中可以在这里调用行情 API、数据库或内部服务
    return {
        "symbol": symbol.upper(),
        "price": 245.67,
```

```
        "timestamp": "2026-05-01 09:30:00"
    }


if __name__ == "__main__":
    # 默认使用 stdio transport，适合本地开发和教学演示
    mcp.run()
```

可以将这个文件保存为：

```
finance_server.py
```

1.2.3 MCP Client（Agent 客户端）

（1）定义

MCP Client 是 Agent 这一侧（即运行 Agent 的程序）用来和 MCP Server 进行通信的“桥梁”和“接口”。 它相当于 Agent 的“工具调用助手”，让 Agent 能够安全、标准地找到并使用 MCP Server 上提供的各种工具。

（2）**主要职责**：

Agent 本身并不直接知道外部有哪些工具可用，也不知道如何正确调用它们。MCP Client 就是为了解决这个问题，它负责：

- 主动去“询问” MCP Server 当前有哪些工具可用
- 帮助 Agent 以正确的格式向 Server 发送调用请求
- 接收 Server 返回的结果，并转换成 Agent 能理解的数据格式
- 处理调用过程中的各种异常情况

简单来说： MCP Client 就是 Agent 与 MCP Server 之间的“翻译官 + 通信员”。

（3）具体工作职责：

1.**连接 MCP Server**

Agent 通过 MCP Client 指定 Server 的启动方式或访问地址，建立连接。

2.**动态发现工具**

Agent 不需要提前硬编码工具名称，而是通过 MCP Client 向 Server 发送“工具发现请求”，Server 会返回当前所有可用的工具列表（包括工具名称、描述、参数要求等）。 这就是 MCP 最强大的特性之一 —— **动态发现**。

3.**构造并发送调用请求**

当 Agent 决定要调用某个工具时，MCP Client 会：

- 把 Agent 提供的参数转换成 MCP 标准格式
- 发送给 Server
- 等待 Server 执行工具

4.**接收并解析返回结果**

Server 执行完工具后，MCP Client 会接收返回结果，并转换成 Python 对象供 Agent 使用。

5.**错误处理**

如果调用失败（权限不足、参数错误、Server 异常等），Client 侧可以捕获异常，方便 Agent 进行针对性处理。

（4）**代码示例**（Agent 侧使用 MCP Client）：

```python
import asyncio

from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client


async def main():
    # 配置 MCP Server 的启动参数
    server_params = StdioServerParameters(
        command="python",
        args=["finance_server.py"]
    )

    # 通过 stdio 启动并连接 MCP Server
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            # 1. 初始化会话
            await session.initialize()

            # 2. 动态发现所有可用工具
            tools_result = await session.list_tools()

            print("可用工具列表：")
            for tool in tools_result.tools:
                print(f"- {tool.name}: {tool.description}")

            # 3. 调用工具
            result = await session.call_tool(
                "get_stock_price",
                arguments={"symbol": "AAPL"}
            )

            print("\n工具调用结果：")
            print(result.content)


if __name__ == "__main__":
    asyncio.run(main())
```

可以将这个文件保存为：

```
finance_client.py
```

运行方式：

```
python finance_client.py
```

（5）**特别说明：server_url 参数**

在一些基于 HTTP、SSE 或 Streamable HTTP 的 MCP Client 封装中，可能会出现 **server_url** 或类似参数。它的含义是：

> **MCP Server 的访问地址**，即 Agent 要连接的“工具服务端”的网络地址。

它类似于我们访问一个网站时的 URL，告诉 Agent：“去哪里找工具？”

但需要注意：

- 在 MCP Python SDK 的基础用法中，Client 通常不是直接通过 `MCPClient(server_url=...)` 连接。
- 不同传输方式有不同的连接方式。
- 本地教学和开发中，最常见的是使用 `stdio_client` 启动并连接本地 MCP Server。
- 远程部署时，可以使用 HTTP、SSE 或 Streamable HTTP 等传输方式。

server_url 的常见格式和使用场景：

本地测试（HTTP 服务模式）

```
server_url="http://localhost:8000"
```

表示连接本机上运行的 MCP Server（端口 8000）。

公司内部服务器（企业实际项目中最常见）

```
server_url="https://mcp.company.com/api"
```

或者

```
server_url="http://192.168.1.100:8000"
```

表示连接公司内部部署的 MCP Server。

云端公共或演示 Server（学习阶段）

```
server_url="https://demo-mcp.example.com"
```

代码示例（说明不同传输方式的差异）：

```python
# stdio 模式：
# Client 通过本地子进程启动并连接 MCP Server
from mcp import StdioServerParameters
from mcp.client.stdio import stdio_client

server_params = StdioServerParameters(
    command="python",
    args=["finance_server.py"]
)

# HTTP / SSE / Streamable HTTP 模式：
# 通常会使用远程服务地址，例如：
server_url = "https://mcp.company.com/api"
```

重要提醒：

- server_url **必须是可访问的地址**，否则 Agent 无法找到工具。
- 在生产环境中，通常使用 HTTPS + 认证机制来保护 server_url 的安全性。
- 在本地实验时，推荐先使用 stdio 模式完成最小可运行示例，再学习 HTTP 或 SSE 等远程部署方式。

1.2.4 MCP 的 Tools 与 LangChain、CrewAI 的 Tools 对比

MCP 的工具体系与我们之前学习的 LangChain.tools 和 CrewAI.tools 属于**不同的设计思路。**

MCP 更强调**标准化、远程调用和生态共享**，而 LangChain/CrewAI 的工具更侧重**本地快速开发。**

以下是三者的详细对比：

对比维度	LangChain.tools / CrewAI.tools	MCP 的 Tools	说明
工具定义方式	使用 `@tool` 装饰器在本地代码中定义	在 MCP Server 中使用标准方式注册工具	MCP 更标准化
工具位置	本地函数，直接运行在 Agent 所在进程	运行在独立的 MCP Server 上（可本地或远程）	MCP 支持解耦部署
调用方式	Agent 直接调用本地函数	Agent 通过 MCP Client 调用	MCP 是协议化调用
动态发现能力	静态定义，无法动态发现	支持运行时动态发现（Agent 可自动获取工具列表）	MCP 优势明显
标准化程度	每个框架格式可能不同	统一工具描述和调用协议，跨客户端通用	MCP 更统一
安全性	依赖框架自身权限控制	可结合认证、权限控制、隔离运行和审计机制	MCP 更适合生产环境
适用场景	本地工具、简单任务、快速开发	企业级工具共享、跨系统集成、生产环境	互补关系

对比维度	LangChain.tools / CrewAI.tools	MCP 的 Tools	说明
复用性	工具仅限当前项目	一次注册，多客户端复用	MCP 优势明显

核心区别总结：

- **LangChain / CrewAI 的 Tools**：更像"本地工具箱"，简单、快速、直接调用，适合学习和小型项目。
- **MCP 的 Tools**：更像"标准化工具服务平台"，强调远程调用、动态发现和安全管控，适合企业级、大规模、跨团队的 Agent 系统。

实际项目中的关系：

在真实开发中，通常是**两者结合使用**：

- 简单、本地、频繁使用的工具 → 使用 `@tool`（LangChain / CrewAI）
- 复杂、企业内部系统、需要权限控制的工具 → 使用 MCP Server 注册

教学小结：

MCP 并不是要替代 LangChain 或 CrewAI 的工具体系，而是提供了一种**更标准化、更安全、更易共享**的工具调用方式。理解两者区别，能帮助我们根据实际需求选择合适的工具管理方案。

2.Skill

2.1 概述

2.1.1 问题导入

在前面的 MCP 部分中，我们已经学习了 Agent 如何通过标准化协议连接外部工具和系统。MCP 解决的是"Agent 如何调用外部能力"的问题。

但在真实任务中，Agent 还会遇到另一个问题：

如何让 Agent 稳定掌握某一类任务的操作方法?

例如，我们希望 Agent 能够完成这些任务：

- 按公司规范撰写周报
- 按固定模板生成竞品分析
- 按指定流程审查合同风险
- 按团队代码规范进行 Code Review
- 按内部 SOP 处理客户投诉

这些任务并不一定都需要复杂工具调用。 很多时候，Agent 缺少的不是工具，而是：

- 不知道该遵循什么流程
- 不知道输出格式应该是什么
- 不知道有哪些业务规则
- 不知道哪些内容不能做

- 不知道遇到异常情况如何处理

如果每次都把这些说明写进用户 Prompt，既冗长又容易遗漏。这时就需要引入 **Skill**。

在许多 Agent 系统中，**Skill 通常以 Markdown 文件的形式存在。** 它像一份“能力说明书”或“操作手册”，作为上下文提供给 Agent，让 Agent 在执行特定任务时可以按既定流程、规则和格式完成工作。

简单来说：

```
Skill = 写给 Agent 看的任务说明书
```

它通常不是一个普通函数，也不一定是一个 API。它更像是一份结构化文档，用自然语言和少量结构化配置告诉 Agent：

```
当你遇到某类任务时，请按照这份说明执行。
```

例如，一个“周报生成 Skill”可以写成 Markdown：

```
# 周报生成 Skill

## 适用场景
当用户要求生成工作周报、项目周报、团队周报时使用本 Skill。

## 输入要求
用户需要提供：
- 本周完成事项
- 下周计划
- 风险和阻塞
- 需要协助的问题

## 执行步骤
1. 整理本周完成事项
2. 按项目或模块归类
3. 提炼关键成果
4. 识别风险和阻塞
5. 生成标准周报

## 输出格式
请按照以下格式输出：

### 本周工作总结
- ...

### 下周工作计划
- ...

### 风险与问题
- ...

### 需要支持
- ...
```

这样，Agent 不需要每次重新“猜”周报怎么写，而是可以直接按照 Skill 中定义的流程完成任务。

因此，在本课程中，我们重点讨论的 Skill 是：

以 Markdown 文档为核心载体、作为上下文注入给 Agent 的任务能力描述。

2.1.2 Skill 的定义

Skill 是一种面向 Agent 的能力描述文件。
它通常以 Markdown 格式编写，用来说明 Agent 在面对某类任务时应该如何理解、规划、执行和输出。

从形式上看，Skill 可以是一份 Markdown 文件：

```
skills/
└── weekly_report/
    └── SKILL.md
```

从内容上看，Skill 通常包括：

- 技能名称
- 适用场景
- 输入要求
- 执行步骤
- 输出格式
- 注意事项
- 禁止行为
- 示例

从运行机制上看，Agent 在需要执行某类任务时，可以将对应 Skill 的 Markdown 内容加载到上下文中。 模型读取 Skill 后，会根据其中的规则和流程完成任务。

可以将 Skill 理解为：

```
Skill = Agent 的可加载上下文能力包
```

与 Tool 不同，Skill 不一定直接执行代码。 它更强调通过上下文指导模型行为。

例如：

类型	主要形式	作用
Tool	函数、API、MCP 工具	执行具体操作
Skill	Markdown 文档	指导 Agent 如何完成一类任务
Memory	历史记录、偏好、知识	提供长期上下文
Workflow	图、状态机、流程编排	控制复杂任务执行路径

Skill 的核心特点是：

- **文本化**：通常用 Markdown 编写
- **上下文化**：作为上下文提供给 Agent
- **任务化**：面向一类任务，而不是一个动作
- **可复用**：可以被多个 Agent 或多个任务复用
- **可维护**：修改 Skill 文档即可调整 Agent 行为

所以，Skill 的本质不是“多写一个函数”，而是**把可复用的任务经验写成 Agent 能理解的上下文规范。**

2.1.3 Skill 与 Prompt 的关系

Skill 和 Prompt 非常接近，因为它们本质上都在影响模型行为。
但二者的粒度和使用方式不同。

对比维度	Prompt	Skill
使用位置	一次对话或一次任务中直接输入	作为可复用能力文件被加载
稳定性	容易随任务变化	相对固定，可版本管理
粒度	可以很小，也可以很临时	面向一类稳定任务
维护方式	写在代码或用户输入中	独立 Markdown 文件维护
复用性	通常较弱	较强，可以被多个 Agent 复用
示例	“请总结这段文本”	“会议纪要生成 Skill”

可以这样理解：

```
Prompt 是一次性的指令。
Skill 是沉淀下来的 Prompt + 流程 + 规范。
```

例如，普通 Prompt 可能是：

```
请帮我把下面会议内容整理成会议纪要。
```

而 Skill 会更完整：

```
# 会议纪要生成 Skill

## 适用场景
当用户提供会议录音转写、会议记录、聊天纪要，并要求整理成正式会议纪要时使用。

## 执行步骤
1. 识别会议主题
2. 提取参会人员
3. 归纳核心讨论内容
4. 提取明确决策
5. 提取待办任务，并标明负责人和截止时间
```

```
6. 对不确定信息进行标注，不要自行编造

## 输出格式
- 会议主题
- 会议时间
- 参会人员
- 核心内容
- 决策事项
- 待办任务
- 风险与待确认事项
```

可以看到，Skill 比普通 Prompt 更系统。
它不是临时告诉模型“做什么”，而是长期告诉模型“这类任务应该怎么做”。

2.1.4 Skill 的核心价值

Skill 的核心价值是：**把稳定任务经验沉淀为可复用上下文。**

（1）减少重复 Prompt 编写

没有 Skill 时，用户或开发者每次都要重复写：

```
请按照以下格式输出.....
请注意不要编造.....
请先判断信息是否完整.....
请把任务拆成.....
```

有了 Skill 后，这些要求可以写进 Markdown 文件中，Agent 在需要时直接加载。

（2）提高 Agent 行为稳定性

同一类任务如果每次都靠模型自由发挥，输出格式和处理流程容易不一致。
Skill 可以固定关键流程，例如：

```
合同审查必须先识别合同类型，再检查付款条款、违约责任、保密条款，最后输出风险等级。
```

这样可以减少模型“想到哪做到哪”的情况。

（3）降低上下文成本

如果把所有规则都写进系统 Prompt，会导致上下文越来越长。
Skill 可以按需加载：

```
用户要写周报 → 加载周报 Skill
用户要审合同 → 加载合同审查 Skill
用户要做代码评审 → 加载 Code Review Skill
```

这比把所有能力说明一次性塞进 Prompt 更高效。

（4）便于版本管理

Skill 是文件，可以像代码一样管理：

```
skills/
├── weekly_report/
|   └── SKILL.md
├── contract_review/
|   └── SKILL.md
└── code_review/
    └── SKILL.md
```

团队可以通过 Git 管理 Skill 的版本：

- 谁修改了规则
- 哪个版本上线
- 哪个 Skill 出现问题
- 如何回滚

（5）让非程序员也能参与建设 Agent

因为 Skill 主要是 Markdown 文档，所以业务专家也能编写和维护。
例如：

- 法务写合同审查 Skill
- 财务写报销审核 Skill
- 销售主管写销售周报 Skill
- 客服主管写投诉处理 Skill

这让 Agent 能力建设不再完全依赖程序员。

2.1.5 Skill 的典型应用场景

Skill 适合封装那些**流程稳定、规则明确、输出格式固定、需要长期复用**的任务。

典型场景包括：

场景	Skill 示例	主要内容
办公自动化	周报生成 Skill、会议纪要 Skill	固定模板、结构化输出、事项整理
企业知识管理	文档摘要 Skill、知识库问答 Skill	摘要规则、引用规范、回答边界
软件研发	Code Review Skill、测试用例生成 Skill	代码规范、检查清单、输出格式
法务合规	合同审查 Skill、隐私风险检查 Skill	风险条款、审查流程、风险等级
客服运营	投诉处理 Skill、工单分类 Skill	分类规则、回复口径、升级条件
数据分析	周报分析 Skill、指标解读 Skill	分析流程、指标口径、报告模板
教学培训	课程讲义生成 Skill、作业批改 Skill	教学风格、评分标准、反馈模板

教学小结：

Skill 最适合沉淀“怎么做”的经验。
Tool 负责“能不能做”，Skill 负责“应该怎么做”。

2.2 Skill 的基本结构

2.2.1 Skill 文件结构

在工程实践中，一个 Skill 通常以目录形式存在。
最核心的文件是 Markdown 文档，例如 `SKILL.md`。

一个典型 Skill 目录可以设计为：

```
skills/
└── weekly_report/
    ├── SKILL.md
    ├── examples/
    |   └── example_01.md
    └── templates/
        └── weekly_report_template.md
```

其中：

- `SKILL.md`：Skill 的核心说明文件
- `examples/`：示例输入和示例输出
- `templates/`：固定输出模板
- `scripts/`：可选脚本文件
- `assets/`：可选资源文件，例如图片、表格、规范文档

最小情况下，一个 Skill 只需要一个 Markdown 文件：

```
skills/
└── meeting_minutes/
    └── SKILL.md
```

这份 Markdown 会作为上下文注入给 Agent。
Agent 读取后，就知道如何处理对应任务。

2.2.2 SKILL.md 的推荐格式

一个高质量的 `SKILL.md` 通常应该包含以下部分：

```
# Skill 名称

## 适用场景
说明什么时候应该使用本 Skill。

## 不适用场景
```

```
说明什么时候不应该使用本 Skill。

## 输入要求
说明用户需要提供哪些信息。

## 执行步骤
说明 Agent 应该如何一步步完成任务。

## 输出格式
规定最终结果的结构和样式。

## 注意事项
说明边界、风险、限制和特殊规则。

## 示例
提供示例输入和示例输出。
```

这不是唯一格式，但非常适合教学和企业落地。
它的核心目标是让 Agent 能清楚回答：

```
我什么时候用这个 Skill？
我需要什么输入？
我应该怎么做？
我最后应该输出什么？
我不能做什么？
```

一个完整例子如下：

```
# 周报生成 Skill

## 适用场景
当用户要求生成个人周报、项目周报、团队周报时，使用本 Skill。

## 不适用场景
以下情况不适合使用本 Skill：
- 用户要求生成日报、月报或年度总结
- 用户没有提供任何工作内容
- 用户要求编造不存在的工作成果

## 输入要求
用户至少需要提供以下信息之一：
- 本周完成事项
- 项目进展
- 问题和风险
- 下周计划

如果信息不足，应先基于已有内容生成简版周报，并明确标注缺失信息。

## 执行步骤
1. 提取用户提供的工作事项
2. 按项目、模块或任务类型进行归类
3. 将零散描述整理成正式表达
```

```
4. 提炼本周关键成果
5. 识别风险、阻塞和待协助事项
6. 整理下周计划
7. 按固定格式输出周报

## 输出格式
请使用以下 Markdown 格式输出：

### 一、本周工作总结
- ...

### 二、关键成果
- ...

### 三、问题与风险
- ...

### 四、下周计划
- ...

### 五、需要支持
- ...

## 注意事项
- 不要编造用户没有提供的工作成果
- 对不确定的信息使用“待确认”标注
- 语言应正式、简洁、适合职场汇报
- 不要输出与周报无关的闲聊内容

## 示例
用户输入：
本周完成了登录模块联调，修了 3 个支付 bug，下周准备做订单导出。

期望输出：
### 一、本周工作总结
- 完成登录模块联调，保障账号登录流程正常运行。
- 修复 3 个支付相关问题，提升支付流程稳定性。

### 二、关键成果
- 登录模块进入可验收状态。
- 支付模块缺陷数量减少，系统稳定性提升。

### 三、问题与风险
- 暂无明确风险，后续需继续关注支付链路回归测试结果。

### 四、下周计划
- 开展订单导出功能开发与联调。

### 五、需要支持
- 暂无。
```

这个 Skill 不包含代码，但它已经能显著提升 Agent 的输出稳定性。

2.2.3 Skill 的元信息

在一些系统中，Skill 还会在 Markdown 顶部使用元信息区，也就是常见的 front matter。
它通常使用 YAML 格式，用来描述 Skill 的名称、版本、描述和适用条件。

例如：

```yaml
---
name: weekly_report
version: 1.0.0
description: 用于根据用户提供的工作内容生成结构化周报
tags:
  - office
  - report
  - writing
---

# 周报生成 Skill

## 适用场景
当用户要求生成个人周报、项目周报、团队周报时，使用本 Skill。
```

这些元信息方便系统做 Skill 检索、筛选、加载和版本管理。

常见字段包括：

字段	含义	示例
name	Skill 唯一名称	`weekly_report`
version	版本号	`1.0.0`
description	简短描述	生成结构化周报
tags	标签	`office`, `report`
author	作者	`training_team`
updated_at	更新时间	`2026-07-24`
trigger	触发条件	用户要求生成周报
permissions	权限要求	普通办公权限

教学小结：

`SKILL.md` 是 Skill 的核心。
它不只是说明文档，而是 Agent 执行任务时会读取的上下文规范。

2.3 Skill 的加载与使用机制

2.3.1 Skill 如何作为 Context 注入

Skill 的关键机制是：**按需加载到上下文中。**

也就是说，Agent 不一定一开始就读取所有 Skill。
通常会先根据用户任务判断应该使用哪个 Skill，然后再把对应的 Markdown 内容加入模型上下文。

流程如下：

```
用户输入任务
    ↓
Agent 或 Router 判断任务类型
    ↓
选择匹配的 Skill
    ↓
读取对应 SKILL.md
    ↓
将 Skill 内容作为上下文注入给模型
    ↓
模型按照 Skill 中的流程和格式执行任务
```

例如用户输入：

```
帮我根据下面内容写一份周报：本周完成了登录模块联调，修复支付 bug。
```

系统判断这是"周报生成"任务，于是加载：

```
skills/weekly_report/SKILL.md
```

最终发送给模型的上下文可能类似：

```
你是一个办公助手。

以下是你需要遵循的 Skill：

[Skill: weekly_report]
# 周报生成 Skill

## 适用场景
...

## 执行步骤
...

## 输出格式
...

用户任务：
帮我根据下面内容写一份周报：本周完成了登录模块联调，修复支付 bug。
```

模型并不是“安装了一个函数”，而是通过阅读这份 Skill 文档，知道该如何执行任务。

这也是为什么 Skill 常常使用 Markdown：
Markdown 对人友好，也对大模型友好。

2.3.2 Skill 选择方式

Agent 如何知道该加载哪个 Skill?
常见方式有三种。

（1）规则匹配

通过关键词或意图规则选择 Skill。

例如：

```python
def select_skill(user_input: str) -> str | None:
    if "周报" in user_input or "工作总结" in user_input:
        return "skills/weekly_report/SKILL.md"

    if "会议纪要" in user_input or "会议记录" in user_input:
        return "skills/meeting_minutes/SKILL.md"

    if "合同" in user_input and "审查" in user_input:
        return "skills/contract_review/SKILL.md"

    return None
```

优点是简单、可控。
缺点是灵活性有限。

（2）向量检索

将 Skill 的名称、描述、标签、适用场景等内容向量化。
当用户输入任务后，通过语义相似度检索最相关的 Skill。

例如：

```
用户输入：帮我整理一下今天会议讨论的内容和待办。
匹配 Skill：meeting_minutes
```

这种方式适合 Skill 数量较多的场景。

（3）LLM Router

让模型根据 Skill 描述选择最合适的 Skill。

例如系统给模型一组候选 Skill：

```
[
  {
    "name": "weekly_report",
    "description": "根据工作内容生成结构化周报"
```

```
  },
  {
    "name": "meeting_minutes",
    "description": "根据会议记录生成会议纪要"
  },
  {
    "name": "contract_review",
    "description": "根据合同文本识别法律风险"
  }
]
```

模型根据用户任务选择：

```
{
  "selected_skill": "meeting_minutes",
  "reason": "用户要求整理会议讨论内容和待办事项"
}
```

在生产环境中，常见做法是组合使用：

```
规则过滤 → 向量召回 → LLM 判断 → 加载 Skill
```

这样既能保证效率，也能提高准确性。

2.3.3 最小可运行示例

下面用一个简单 Python 示例模拟 Skill 的加载过程。
注意：这个例子不依赖具体 Agent 框架，重点演示“读取 Markdown Skill 并作为上下文拼接”。

假设目录结构如下：

```
project/
├── main.py
└── skills/
    └── weekly_report/
        └── SKILL.md
```

`skills/weekly_report/SKILL.md` 内容如下：

```
# 周报生成 Skill

## 适用场景
当用户要求生成工作周报、项目周报、团队周报时，使用本 Skill。

## 执行步骤
1. 提取用户提供的工作事项
2. 按项目或模块归类
3. 整理成本周工作总结
4. 提炼关键成果
5. 识别问题与风险
```

```
6. 生成下周计划

## 输出格式
请按照以下格式输出：

### 一、本周工作总结
- ...

### 二、关键成果
- ...

### 三、问题与风险
- ...

### 四、下周计划
- ...

## 注意事项
- 不要编造用户未提供的信息
- 信息不足时标注“待补充”
- 语言正式、简洁
```

`main.py` 示例：

```python
from pathlib import Path


def load_skill(skill_path: str) -> str:
    """读取 Skill Markdown 文件"""
    return Path(skill_path).read_text(encoding="utf-8")


def build_prompt(user_input: str, skill_content: str) -> str:
    """将 Skill 内容拼接到模型上下文中"""
    return f"""
你是一个企业办公助手。

请严格遵循以下 Skill 完成任务：

{skill_content}

用户任务：
{user_input}
"""


def select_skill(user_input: str) -> str | None:
    """根据用户输入选择 Skill"""
    if "周报" in user_input or "工作总结" in user_input:
        return "skills/weekly_report/SKILL.md"

    return None

```

```python

if __name__ == "__main__":
    user_input = "帮我写一份周报：本周完成了登录模块联调，修复了 3 个支付 bug，下周做订单导出。"

    skill_path = select_skill(user_input)

    if skill_path is None:
        prompt = user_input
    else:
        skill_content = load_skill(skill_path)
        prompt = build_prompt(user_input, skill_content)

    print(prompt)
```

运行后可以看到，Skill 文档会被拼接进 Prompt 中。
在真实 Agent 系统中，这个 `prompt` 会继续发送给 LLM，由模型根据 Skill 完成任务。

如果接入 OpenAI、Claude、Gemini 或本地模型，流程本质也是一样的：

```
读取 Skill Markdown → 拼接上下文 → 调用模型 → 生成结果
```

2.4 Skill 的编写规范

2.4.1 编写高质量 Skill 的原则

Skill 写得好不好，会直接影响 Agent 的行为稳定性。
一个高质量 Skill 应遵循以下原则。

（1）任务边界清晰

Skill 必须明确说明适用场景和不适用场景。
否则 Agent 容易误用。

例如：

```markdown
## 适用场景
当用户要求生成个人周报、项目周报、团队周报时使用。

## 不适用场景
- 用户要求生成年度总结
- 用户要求编造不存在的工作成果
- 用户只是询问“什么是周报”
```

（2）步骤明确

执行步骤应该具体，不要只写抽象原则。

不推荐：

```markdown
## 执行步骤
请认真分析用户内容，并生成高质量周报。
```

推荐：

```
## 执行步骤
1. 提取所有工作事项
2. 按项目或模块分类
3. 将口语化内容改写成正式表达
4. 提炼关键成果
5. 标注问题与风险
6. 生成下周计划
```

（3）输出格式固定

Skill 应该明确规定输出结构。
这可以显著提高结果一致性。

例如：

```
## 输出格式

### 一、本周工作总结
- ...

### 二、关键成果
- ...

### 三、问题与风险
- ...

### 四、下周计划
- ...
```

（4）禁止编造

很多 Skill 都应强调“不要编造”。
尤其是报告、会议纪要、法务、财务、医疗等场景。

例如：

```
## 注意事项
- 不要编造用户没有提供的信息
- 对缺失内容标注“待补充”
- 对不确定内容标注“待确认”
```

（5）给出示例

示例是提高 Skill 效果的重要方式。
对模型来说，示例比抽象描述更直观。

例如：

```
## 示例

用户输入：
本周完成登录模块联调，修复 3 个支付 bug。

期望输出：
### 一、本周工作总结
- 完成登录模块联调，保障账号登录流程正常运行。
- 修复 3 个支付相关问题，提升支付流程稳定性。
```

（6）控制长度

Skill 不是越长越好。
太长会增加上下文成本，也可能让模型抓不住重点。

推荐原则：

- 简单 Skill：300 ~ 800 字
- 中等 Skill：800 ~ 1500 字
- 复杂 Skill：可以拆分为多个 Skill 或引用模板文件

（7）使用清晰的 Markdown 结构

Markdown 标题、列表、代码块、表格都可以帮助模型理解结构。

推荐格式：

```
# Skill 名称

## 适用场景
...

## 输入要求
...

## 执行步骤
...

## 输出格式
...

## 注意事项
...

## 示例
...
```

2.4.2 常见 Skill 模板

下面给出几个常见 Skill 模板，可以直接作为课程示例。

（1）会议纪要 Skill

```
# 会议纪要生成 skill

## 适用场景
当用户提供会议记录、会议转写文本、群聊讨论内容，并要求整理成正式会议纪要时使用。

## 输入要求
用户应提供会议相关文本。若缺少会议时间、参会人员等信息，应标注“待补充”。

## 执行步骤
1. 识别会议主题
2. 提取参会人员和会议时间
3. 归纳核心讨论内容
4. 提取明确决策
5. 提取待办事项
6. 为每个待办事项标注负责人和截止时间
7. 对不确定信息标注“待确认”

## 输出格式

### 会议主题
...

### 会议时间
...

### 参会人员
...

### 核心讨论内容
- ...

### 决策事项
- ...

### 待办事项
| 事项 | 负责人 | 截止时间 | 状态 |
| ---- | ------ | -------- | ---- |
| ... | ... | ... | ... |

### 待确认问题
- ...

## 注意事项
- 不要编造参会人员、时间、决策事项
- 如果负责人或截止时间未出现，填写“待确认”
- 语言应正式、简洁
```

（2）Code Review Skill

```
# Code Review Skill

## 适用场景
当用户提交代码片段，并要求进行代码审查、质量检查、安全检查或优化建议时使用。

## 输入要求
用户需要提供代码片段，并尽量说明编程语言、业务背景和关注点。

## 执行步骤
1. 判断代码语言和主要功能
2. 检查代码是否存在明显 bug
3. 检查命名、结构和可读性
4. 检查性能问题
5. 检查安全风险
6. 给出可执行修改建议
7. 如有必要，提供改进后的代码片段

## 输出格式

### 代码功能概述
...

### 主要问题
| 问题类型 | 位置 | 说明 | 严重程度 |
| -------- | ---- | ---- | -------- |
| ... | ... | ... | ... |

### 优化建议
- ...

### 改进示例
```python
# 示例代码
```

注意事项

- 不要无依据地指出不存在的问题
- 对安全问题要明确说明风险原因
- 修改建议应尽量具体、可执行

```
注意：如果在 Markdown Skill 中嵌套代码块，实际文件里可以使用更多反引号避免代码块冲突。

（3）合同审查 Skill

```markdown
# 合同审查 Skill

## 适用场景
```

```
当用户提供合同文本，并要求识别风险条款、审查合同问题或生成法务审查意见时使用。

## 输入要求
用户需要提供合同正文。若合同文本不完整，应说明审查结果仅基于已提供内容。

## 执行步骤
1. 识别合同类型
2. 提取合同双方、金额、期限、付款方式等关键信息
3. 检查付款条款是否明确
4. 检查违约责任是否对等
5. 检查保密条款和知识产权条款
6. 检查争议解决方式
7. 识别高风险或模糊条款
8. 输出风险等级和修改建议

## 输出格式

### 合同基本信息
| 字段 | 内容 |
| ---- | ---- |
| 合同类型 | ... |
| 甲方 | ... |
| 乙方 | ... |
| 金额 | ... |
| 履行期限 | ... |

### 风险摘要
- ...

### 风险明细
| 条款 | 风险说明 | 风险等级 | 修改建议 |
| ---- | -------- | -------- | -------- |
| ... | ... | 高/中/低 | ... |

### 总体建议
...

## 注意事项
- 不要编造合同中不存在的条款
- 如果合同内容缺失，应明确说明限制
- 不输出确定性法律结论，只提供审查建议
- 高风险合同应建议人工法务复核
```

2.5 Skill、Tool 与 MCP 的关系

2.5.1 三者的区别

Skill、Tool 和 MCP 经常一起出现，但它们解决的问题不同。

维度	Skill	Tool	MCP
本质	Markdown 能力说明 / 上下文规范	函数、API、可执行动作	工具连接协议
主要作用	告诉 Agent “怎么做”	帮 Agent “做具体动作”	标准化连接外部工具
是否执行代码	通常不直接执行	会执行	本身是协议，不直接等于业务逻辑
典型形式	`SKILL.md`	Python 函数、HTTP API、MCP Tool	Client-Server 协议
示例	周报生成 Skill	读取销售数据工具	数据库 MCP Server

可以这样理解：

```
Skill 负责方法。
Tool 负责动作。
MCP 负责连接。
```

例如“生成销售周报”：

```
Skill：
规定销售周报怎么写、分析哪些指标、输出什么格式

Tool：
查询销售数据、计算增长率、生成图表

MCP：
让 Agent 标准化连接数据库、报表系统、文档系统
```

三者不是互斥关系，而是可以组合使用。

2.5.2 三者如何协同

一个完整的高阶 Agent 系统中，Skill、Tool 和 MCP 可以这样协同：

```
用户：请生成上周华东区销售周报
    ↓
Agent 判断任务类型
    ↓
加载 weekly_report Skill
    ↓
```

```
Skill 告诉 Agent：
- 需要哪些数据
- 按什么步骤分析
- 用什么格式输出
    ↓
Agent 调用 Tool 查询数据
    ↓
Tool 通过 MCP 连接企业数据库
    ↓
Agent 根据 Skill 格式生成周报
```

对应关系如下：

层级	作用	示例
Agent	理解用户意图，选择 Skill 和 Tool	判断用户要生成销售周报
Skill	作为上下文指导任务流程	`weekly_report/SKILL.md`
Tool	执行具体动作	查询销售额、导出图表
MCP	标准化连接工具和系统	PostgreSQL MCP Server

教学小结：

Skill 并不是 MCP Server，也不是普通函数。
它更接近一种“Agent 可读取的任务说明文档”。

在高阶 Agent 中：

- **MCP** 解决“工具怎么接入”
- **Tool** 解决“具体动作怎么执行”
- **Skill** 解决“任务应该怎么完成”
- **Agent** 负责“什么时候用哪个能力”

Skill 的引入，让 Agent 不只是临时推理，而是能够复用沉淀好的任务经验。

3.LangGraph

3.1 概述

3.1.1 问题导入

在前面的课程中，我们已经学习了多种构建 Agent 的方式：

- 使用 LangChain 的 `create_agent` 快速创建单 Agent
- 使用 CrewAI 的 `Crew` 组织多个 Agent 进行团队协作
- 使用 Workflow 将复杂任务拆成多个明确步骤
- 使用 MCP 让 Agent 标准化连接外部工具

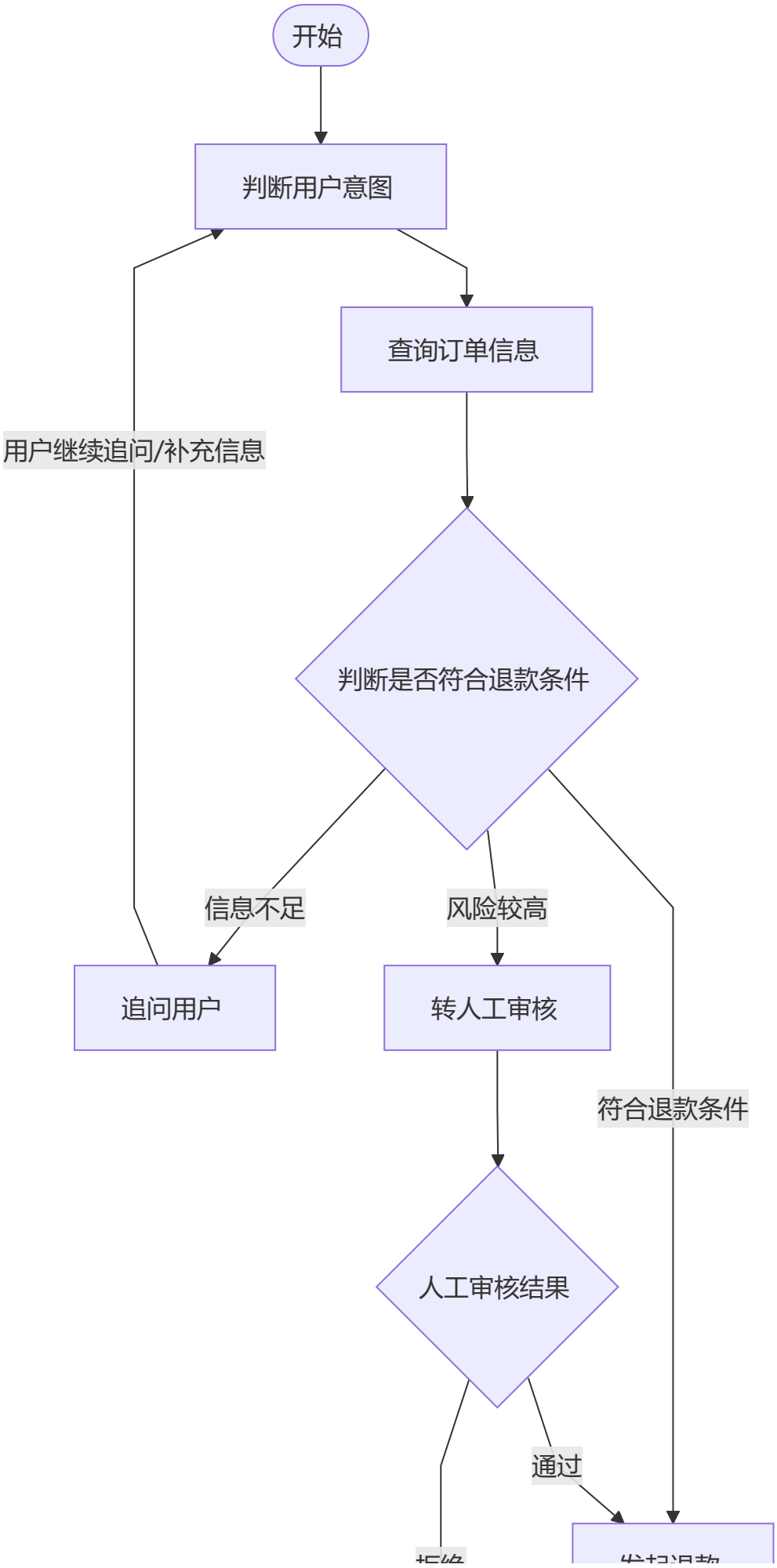
- 使用 Skill 将稳定任务经验沉淀成可复用上下文

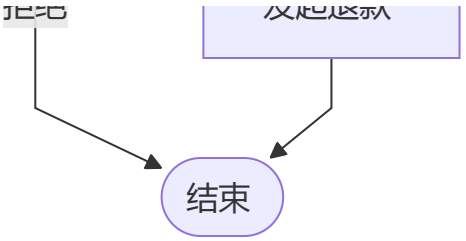
这些内容已经可以帮助我们构建出一个“能工作的 Agent 系统”。
但当 Agent 进入真实生产环境时，一个新问题会逐渐变得明显：

简单 Agent 和线性 Workflow 很难处理复杂、动态、可恢复的任务流程。

例如，一个客服处理 Agent 可能需要完成：

```
1. 判断用户意图
2. 查询订单信息
3. 判断是否符合退款条件
4. 如果信息不足，追问用户
5. 如果符合规则，发起退款
6. 如果风险较高，转人工审核
7. 如果用户继续追问，回到前面重新判断
8. 全程记录状态，支持中断后恢复
```


开始
判断用户意图
查询订单信息
用户继续追问/补充信息
判断是否符合退款条件
信息不足
风险较高
追问用户
转人工审核
符合退款条件
人工审核结果
通过



这已经不是简单的“工具调用”或“顺序流程”了。
它更像一个有状态的任务图：

- 有分支
- 有循环
- 有状态
- 有中断
- 有恢复
- 有人工介入
- 有多 Agent 协作
- 有长期运行任务

这时，就需要引入 **LangGraph**。

LangGraph 可以理解为 Workflow 的高阶升级版。
它不是简单地把步骤排成一条直线，而是把 Agent 任务建模成一个“图结构”。
图中的每个节点负责一个处理步骤，边负责决定下一步去哪里，状态负责记录整个任务过程中的上下文。

简单来说：

```
LangGraph = 有状态的 Agent 工作流图
```

如果说 `create_agent` 适合快速创建一个能调用工具的 Agent，CrewAI 适合组织多个角色协作，普通 Workflow 适合稳定的线性流程，那么 LangGraph 更适合构建复杂、可控、可恢复的生产级 Agent 系统。

3.1.2 LangGraph 的定义

LangGraph 是 LangChain 生态中的一个用于构建有状态、多步骤、可循环 Agent 应用的框架。
它的核心思想是：将 Agent 的执行过程建模为一个图。

在这个图中：

- **State** 表示任务执行过程中的共享状态
- **Node** 表示一个具体处理步骤
- **Edge** 表示步骤之间的流转关系
- **Conditional Edge** 表示根据条件动态选择下一步
- **Graph** 表示完整任务流程

可以用一个简单结构表示：

```
State
 ↓
Node A
 ↓
根据条件判断
 ├── Node B
 ├── Node C
 └── END
```

与普通 Workflow 不同，LangGraph 不要求流程只能从头到尾线性执行。
它可以支持：

- 条件分支
- 循环重试
- 多节点协作
- 人工确认
- 中断恢复
- 状态持久化

这让 LangGraph 非常适合高阶 Agent 场景。

例如，普通 Workflow 可能是：

```
查询资料 → 总结内容 → 生成报告
```

而 LangGraph 可以是：

```
理解任务
 ↓
判断资料是否足够
 ├── 足够 → 生成报告
 └── 不足 → 继续检索资料 → 再次判断
```

这就是从“流程”到“图”的升级。

3.1.3 LangGraph 与 create_agent、CrewAI、Workflow 的关系

LangGraph 并不是要替代前面学过的所有内容，而是提供一种更强的编排方式。
它和 `create_agent`、CrewAI、Workflow 是递进关系。

能力形态	主要特点	适合场景
`create_agent`	快速创建单个 Agent，支持工具调用	简单助手、单轮任务、快速原型
CrewAI `Crew`	多 Agent 角色协作，强调团队分工	内容生产、研究分析、角色协作任务
普通 Workflow	步骤明确，流程线性或半线性	固定 SOP、审批流程、标准任务

能力形态	主要特点	适合场景
LangGraph	有状态、可分支、可循环、可恢复	复杂 Agent、生产级流程、动态决策任务

可以这样理解：

```
create_agent：先让一个 Agent 能工作
CrewAI：让多个 Agent 像团队一样协作
Workflow：让任务按照固定流程稳定执行
LangGraph：让复杂任务在状态图中动态流转
```

它们之间不是互斥关系。
在实际项目中，LangGraph 的某个节点里可以调用：

- 一个普通工具
- 一个 MCP Tool
- 一个 Skill
- 一个 `create_agent` 创建的 Agent
- 一个 CrewAI 团队
- 一个 RAG 检索流程

例如：

```
LangGraph
  ├── 节点 A：理解用户意图
  ├── 节点 B：调用 Skill 生成分析思路
  ├── 节点 C：调用 MCP Tool 查询数据库
  ├── 节点 D：调用 RAG 检索资料
  ├── 节点 E：调用 Agent 生成最终回答
  └── 节点 F：人工审核
```

因此，LangGraph 更像是高阶 Agent 系统中的“流程骨架”和“状态控制器”。

3.1.4 LangGraph 的核心价值

LangGraph 的核心价值在于：**让 Agent 的复杂执行过程变得可控、可追踪、可恢复。**

（1）支持复杂流程编排

普通链式调用通常只能处理：

```
A → B → C
```

而 LangGraph 可以处理：

```
A → 判断
    ├── B → D
    ├── C → D
    └── E → 人工审核
```

这非常适合真实业务场景。

（2）支持循环和重试

Agent 常常需要反复尝试，例如：

- 检索结果不够好 → 继续检索
- 代码运行失败 → 修复代码再运行
- 用户信息不足 → 追问用户
- 生成答案不合格 → 重新生成

LangGraph 可以通过条件边实现循环。

```
生成答案 → 质量检查
    ├── 通过 → 输出
    └── 不通过 → 重新生成
```

（3）支持状态管理

LangGraph 的每一步都可以读写共享状态。
这让系统可以清楚知道当前任务进展：

```
{
    "user_input": "帮我分析这份销售数据",
    "intent": "sales_analysis",
    "retrieved_docs": [...],
    "analysis_result": "...",
    "need_human_review": False
}
```

有了状态，Agent 就不再只是“边聊边忘”，而是可以基于持续上下文推进任务。

（4）支持中断与恢复

生产环境中，很多任务不是一次完成的。
例如：

- 等待用户补充信息
- 等待人工审批
- 等待外部系统返回结果
- 长任务执行到一半需要暂停

LangGraph 可以配合 checkpoint 机制保存状态，让任务之后继续执行。

（5）更适合生产环境

LangGraph 把 Agent 过程显式建模成图。
这让系统更容易：

- 调试
- 监控
- 测试
- 审计
- 优化
- 灰度上线

这也是它相比“全靠 Prompt 让 Agent 自己发挥”的关键优势。

3.2 LangGraph 的核心概念

3.2.1 State、Node、Edge、Graph

理解 LangGraph，最重要的是掌握四个核心概念：**State、Node、Edge、Graph。**

（1）State：状态

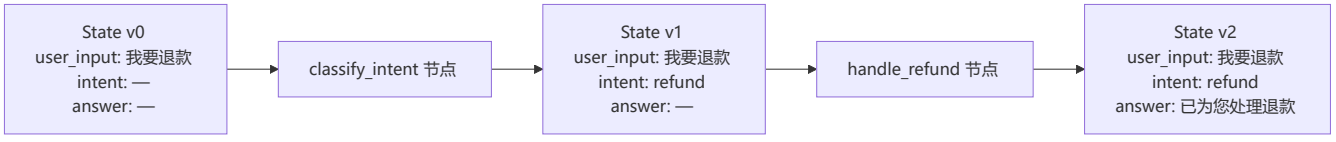
State 是整个图运行过程中共享的数据。每个节点都可以读取 State，也可以向 State 写入新的内容。

例如：

```python
from typing import TypedDict


class AgentState(TypedDict):
    user_input: str
    intent: str
    answer: str
```

可以把 State 理解为“任务档案袋”。每一步处理完，都把结果放进这个档案袋里，后续节点继续使用。



（2）Node：节点

Node 是图中的一个处理步骤。它通常是一个函数，接收 State，返回需要更新的 State 字段。

例如：

```python
def classify_intent(state: AgentState):
    user_input = state["user_input"]

    if "退款" in user_input:
        intent = "refund"
    else:
        intent = "general"

    return {
        "intent": intent
    }
```

这个节点的作用是识别用户意图。

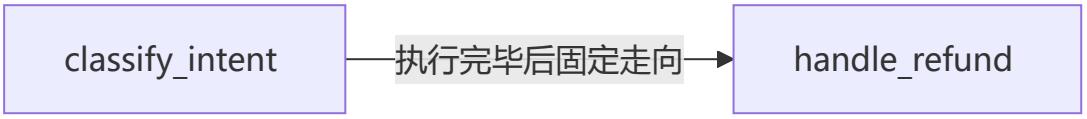


（3）Edge：边

Edge 表示节点之间的连接关系。它决定一个节点执行完之后，下一步执行哪个节点。

例如：

```
classify_intent → handle_refund
```

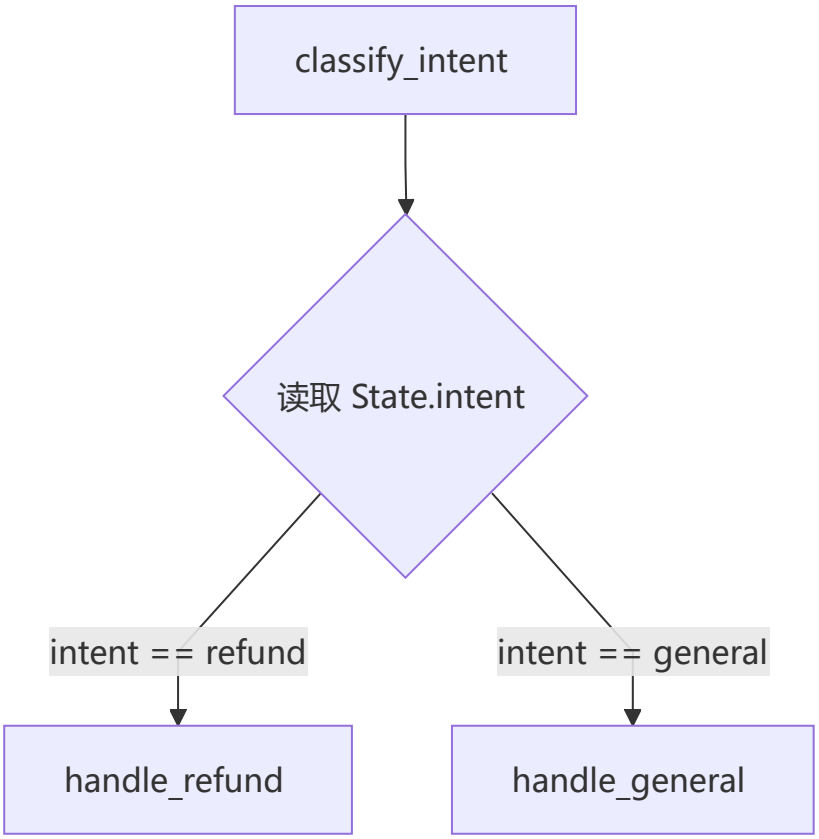


（4）Conditional Edge：条件边

条件边可以根据 State 动态选择下一步。

例如：

```
如果 intent = refund → 进入退款流程
如果 intent = general → 进入普通问答流程
```

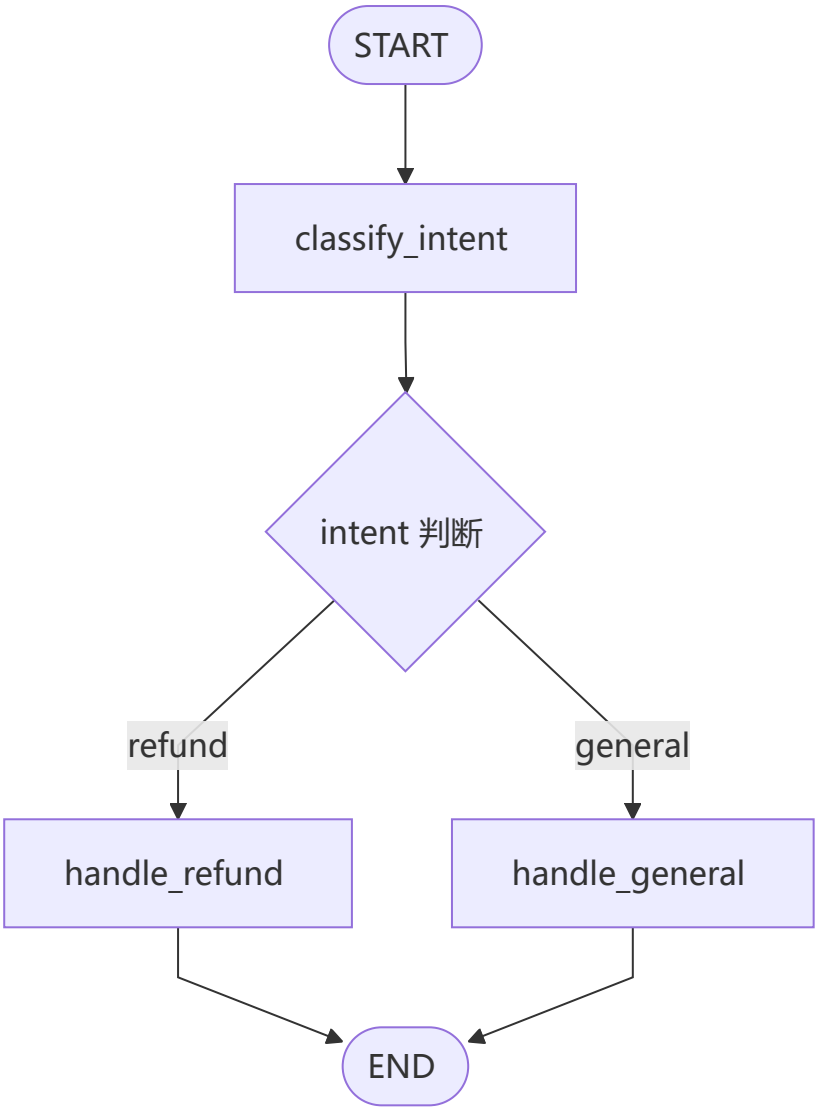


（5）Graph：图

Graph 是完整流程。它由 State、Node 和 Edge 组合而成。

一个简单图可以表示为：

```
START
 ↓
classify_intent
 ↓
根据 intent 判断
 ├── handle_refund
 └── handle_general
 ↓
END
```



3.2.2 最小可运行示例

下面用一个最小示例演示 LangGraph 的基本用法。这个例子实现一个简单意图识别流程：

```
用户输入
 ↓
识别意图
 ↓
根据意图选择处理节点
 ↓
输出结果
```

安装依赖：

```
pip install langgraph
```

代码示例：

```python
from typing import TypedDict

from langgraph.graph import StateGraph, START, END


class AgentState(TypedDict):
    user_input: str
    intent: str
    answer: str


def classify_intent(state: AgentState):
    user_input = state["user_input"]

    if "退款" in user_input:
        intent = "refund"
    else:
        intent = "general"

    return {
        "intent": intent
    }


def handle_refund(state: AgentState):
    return {
        "answer": "您正在咨询退款问题，请提供订单号，我将继续帮您处理。"
    }


def handle_general(state: AgentState):
    return {
        "answer": "您好，我已经收到您的问题，将为您提供通用帮助。"
    }


def route_by_intent(state: AgentState):
    if state["intent"] == "refund":
        return "refund"

    return "general"


# 1. 创建图
builder = StateGraph(AgentState)

# 2. 添加节点
builder.add_node("classify_intent", classify_intent)
builder.add_node("handle_refund", handle_refund)
builder.add_node("handle_general", handle_general)

# 3. 添加边
builder.add_edge(START, "classify_intent")
```

```python

# 4. 添加条件边
builder.add_conditional_edges(
    "classify_intent",
    route_by_intent,
    {
        "refund": "handle_refund",
        "general": "handle_general"
    }
)

builder.add_edge("handle_refund", END)
builder.add_edge("handle_general", END)

# 5. 编译图
graph = builder.compile()

# 6. 执行图
result = graph.invoke({
    "user_input": "我想申请退款",
    "intent": "",
    "answer": ""
})

print(result["answer"])
```

运行结果类似：

```
您正在咨询退款问题，请提供订单号，我将继续帮您处理。
```

这个例子虽然简单，但已经体现了 LangGraph 的核心思想：

- 用 State 保存任务状态
- 用 Node 表示处理步骤
- 用 Conditional Edge 表示动态分支
- 用 Graph 管理完整流程

3.2.3 从普通 Workflow 升级到 LangGraph

我们前面已经学习过 Workflow，普通 Workflow 通常是这样的：

```python
def workflow(user_input: str):
    intent = classify_intent(user_input)

    if intent == "refund":
        answer = handle_refund(user_input)
    else:
        answer = handle_general(user_input)

    return answer
```

这种写法在简单场景中完全可用，但随着流程变复杂，会出现几个问题：

- 状态分散在多个变量中
- 分支越来越多，代码越来越难维护
- 循环、重试和人工介入难以表达
- 很难记录每一步执行过程
- 中断后不容易恢复

LangGraph 的升级点在于，它把这些流程显式建模：

```
函数里的隐式流程
        ↓
图里的显式节点和边
```

对比来看：

普通 Workflow	**LangGraph**
流程写在函数内部	流程显式表示为图
状态靠变量传递	状态统一由 State 管理
分支靠 if/else	分支由条件边管理
循环不直观	图天然支持循环
难以中断恢复	可结合 checkpoint 持久化
调试粒度较粗	每个节点都可单独观察

所以，LangGraph 并不是让简单问题复杂化，而是在任务复杂度上升后，让系统结构仍然清晰。

3.3 LangGraph 的典型模式

3.3.1 条件分支模式

条件分支是 LangGraph 最常见的用法，它适合根据用户意图、任务状态或工具结果选择不同路径。

例如客服场景：

```
识别用户意图
  ↓
条件判断
  ├── 退款问题 → 退款处理节点
  ├── 物流问题 → 物流查询节点
  ├── 投诉问题 → 投诉升级节点
  └── 普通问题 → 通用回答节点
```

这种模式比把所有逻辑都写在一个大 Prompt 里更可控，每条路径都可以单独设计、测试和优化。

3.3.2 循环重试模式

循环重试适合需要不断改进结果的任务。

例如 RAG 问答：

```
用户问题
  ↓
检索资料
  ↓
判断资料是否足够
  ├── 足够 → 生成回答
  └── 不足 → 改写查询 → 再次检索
```

再比如代码生成：

```
生成代码
  ↓
运行测试
  ↓
判断是否通过
  ├── 通过 → 输出代码
  └── 不通过 → 修复代码 → 再次运行测试
```

LangGraph 的优势是可以明确控制循环条件和最大次数，避免 Agent 无限尝试。

3.3.3 Human-in-the-loop 模式

在真实生产环境中，并不是所有任务都应该完全自动化，对于高风险任务，应该加入人工确认。

例如：

```
Agent 生成退款方案
 ↓
判断退款金额
 ├── 小额退款 → 自动处理
 └── 大额退款 → 人工审核
```

适合加入人工确认的场景包括：

- 大额退款
- 合同审批
- 数据删除
- 外部邮件发送
- 财务报销
- 法务风险判断

LangGraph 可以将“等待人工确认”设计成流程中的一个节点，让系统既智能又安全。

3.3.4 多 Agent 编排模式

前面我们学习过 CrewAI 的 Crew 团队协作，LangGraph 也可以用来编排多个 Agent，只是它更强调流程控制和状态流转。

例如一个研究报告生成流程：

```
规划 Agent
 ↓
检索 Agent
 ↓
分析 Agent
 ↓
写作 Agent
 ↓
审核 Agent
 ↓
最终报告
```

也可以加入条件判断：

```
审核 Agent 判断质量
 ├── 通过 → 输出最终报告
 └── 不通过 → 返回分析 Agent 重新分析
```

与 CrewAI 相比，LangGraph 的特点是：

- 更强调显式流程
- 更适合复杂分支
- 更容易控制循环
- 更适合生产环境中的状态管理

3.4 LangGraph 综合案例：客服退款 Agent

3.4.1 案例目标

前面我们已经学习了 LangGraph 的几种典型模式：

- **条件分支模式**：根据任务状态选择不同路径
- **循环重试模式**：当结果不满足要求时重新执行
- **Human-in-the-loop 模式**：高风险任务引入人工确认
- **多 Agent / LLM 编排模式**：在节点中调用 LLM 或 Agent 完成复杂判断

下面我们通过一个完整案例，把这些能力组合起来。

本案例要实现一个“客服退款 Agent”，它可以完成以下流程：

```
用户提出问题
 ↓
使用 create_agent 创建的 Agent 判断用户意图
 ↓
条件分支：
 ├── 退款问题 → 进入退款流程
 └── 普通问题 → 直接回答
 ↓
退款流程中检查订单信息
 ↓
如果订单信息不足：
 └── 循环追问用户，直到信息足够或达到最大次数
 ↓
判断退款金额
 ↓
条件分支：
 ├── 小额退款 → 自动生成处理结果
 └── 大额退款 → 进入人工确认 Human-in-the-loop
 ↓
输出最终结果
```

可以用图表示为：

```
START
  ↓
意图识别 Agent
  ↓
是否退款问题？
  ├── 否 → 普通客服回复 → END
```

```
  |
  └── 是
        ↓
    检查订单信息
        ↓
    信息是否完整？
        ├── 否 → 追问用户 → 再次检查订单信息
        |
        └── 是
              ↓
          判断退款金额
              ↓
          是否大额退款？
              ├── 否 → 自动退款处理 → END
              |
              └── 是 → 人工确认 → END
```

这个案例的重点不是实现真实退款接口，而是帮助学生理解：
LangGraph 如何把 Agent、LLM、条件分支、循环和人工确认组织成一个可控流程。

3.4.2 环境准备

本案例使用：

- `langgraph`：构建流程图
- `langchain`：使用 `create_agent`
- `langchain-openai`：使用 `ChatOpenAI`

安装依赖：

```
pip install langgraph langchain langchain-openai
```

3.4.3 完整代码示例

下面是完整可运行示例。为了方便教学，案例中没有真正连接订单系统，而是用简单规则模拟订单检查和退款金额判断。

```python
import os
from typing import TypedDict, Literal

from langchain_openai import ChatOpenAI
from langchain.agents import create_agent

from langgraph.graph import StateGraph, START, END


# 如果没有在系统环境变量中配置，可以临时打开下面这一行
# os.environ["OPENAI_API_KEY"] = "你的APIKey"
```

```python


class CustomerServiceState(TypedDict):
    user_input: str
    intent: str
    order_id: str
    refund_amount: float
    retry_count: int
    max_retry: int
    need_human_review: bool
    human_approved: bool
    answer: str


# 1. 创建 LLM
llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0
)


# 2. 使用 create_agent 创建一个意图识别 Agent
intent_agent = create_agent(
    model=llm,
    tools=[],
    system_prompt="""
你是一个客服意图识别 Agent。

你的任务是判断用户输入是否属于退款问题。

请只输出以下两个标签之一：
- refund：用户想要退款、退货、取消订单并返还金额
- general：其他普通客服问题

不要输出解释。
"""
)


def classify_intent(state: CustomerServiceState):
    """使用 create_agent 判断用户意图"""

    result = intent_agent.invoke({
        "messages": [
            {
                "role": "user",
                "content": state["user_input"]
            }
        ]
    })

    # create_agent 返回结果中通常包含 messages
    last_message = result["messages"][-1]
```

```python
    intent = last_message.content.strip().lower()

    if "refund" in intent:
        intent = "refund"
    else:
        intent = "general"

    return {
        "intent": intent
    }


def route_by_intent(state: CustomerServiceState) -> Literal["refund", "general"]:
    """根据意图选择流程"""

    if state["intent"] == "refund":
        return "refund"

    return "general"


def handle_general_question(state: CustomerServiceState):
    """处理普通客服问题"""

    return {
        "answer": "您好，您的问题已收到。这是一个普通客服问题，我会为您提供常规帮助。"
    }


def check_order_info(state: CustomerServiceState):
    """
    检查订单信息是否完整。

    教学简化：
    - 如果用户输入中包含 ORD，则认为包含订单号
    - 如果没有订单号，则要求用户补充
    """

    user_input = state["user_input"]

    if "ORD" in user_input:
        # 教学中简单模拟订单号和退款金额
        # 真实项目中这里应调用订单查询 Tool 或 MCP Server
        order_id = "ORD-20260724-001"

        if "大额" in user_input or "5000" in user_input:
            refund_amount = 5000.0
        else:
            refund_amount = 99.0

        return {
            "order_id": order_id,
            "refund_amount": refund_amount,
```

```
            "answer": "已识别到订单信息，正在继续判断退款规则。"
        }

    return {
        "answer": "为了继续处理退款，请您提供订单号，例如 ORD-20260724-001。"
    }


def route_by_order_info(state: CustomerServiceState) -> Literal["complete", "incomplete", "failed"]:
    """根据订单信息是否完整选择下一步"""

    if state["order_id"]:
        return "complete"

    if state["retry_count"] >= state["max_retry"]:
        return "failed"

    return "incomplete"


def ask_for_order_info(state: CustomerServiceState):
    """
    循环追问用户补充订单信息。

    为了课堂演示，这里使用 input() 模拟用户补充信息。
    在真实 Web / API 系统中，这一步通常会中断流程，等待用户下一轮输入。
    """

    print("\nAgent：为了继续处理退款，请提供订单号。")
    new_input = input("用户补充：")

    return {
        "user_input": state["user_input"] + "\n" + new_input,
        "retry_count": state["retry_count"] + 1
    }


def handle_missing_order_info(state: CustomerServiceState):
    """达到最大追问次数后结束流程"""

    return {
        "answer": "由于多次未能获取有效订单号，暂时无法继续处理退款申请，请稍后重新提交完整订单信息。"
    }


def judge_refund_amount(state: CustomerServiceState):
    """
    判断是否需要人工审核。

    教学规则：
    - 退款金额大于等于 1000 元，需要人工审核
```

```
    - 否则自动处理
    """

    refund_amount = state["refund_amount"]

    if refund_amount >= 1000:
        return {
            "need_human_review": True,
            "answer": f"该订单退款金额为 {refund_amount} 元，属于大额退款，需要人工审核。"
        }

    return {
        "need_human_review": False,
        "answer": f"该订单退款金额为 {refund_amount} 元，可进入自动退款流程。"
    }


def route_by_refund_amount(state: CustomerServiceState) -> Literal["auto", "human"]:
    """根据退款金额选择自动处理或人工审核"""

    if state["need_human_review"]:
        return "human"

    return "auto"


def auto_refund(state: CustomerServiceState):
    """自动退款处理"""

    return {
        "answer": f"订单 {state['order_id']} 已通过自动退款规则，退款金额
{state['refund_amount']} 元，预计 1-3 个工作日原路返回。"
    }


def human_review(state: CustomerServiceState):
    """
    Human-in-the-loop：人工确认。

    为了课堂演示，使用 input() 模拟人工审核。
    真实生产系统中，这里通常会进入审批流、工单系统或后台审核页面。
    """

    print("\n系统提示：该退款申请需要人工审核。")
    print(f"订单号：{state['order_id']}")
    print(f"退款金额：{state['refund_amount']}")

    decision = input("人工审核是否通过？请输入 yes / no：").strip().lower()

    approved = decision == "yes"

    return {
        "human_approved": approved
```

```python
    }


def handle_human_result(state: CustomerServiceState):
    """根据人工审核结果生成最终回复"""

    if state["human_approved"]:
        return {
            "answer": f"订单 {state['order_id']} 的大额退款申请已通过人工审核，退款金额 {state['refund_amount']} 元，后续将进入退款流程。"
        }

    return {
        "answer": f"订单 {state['order_id']} 的大额退款申请未通过人工审核，建议联系人工客服了解具体原因。"
    }


# 3. 构建 LangGraph
builder = StateGraph(CustomerServiceState)

# 添加节点
builder.add_node("classify_intent", classify_intent)
builder.add_node("handle_general_question", handle_general_question)
builder.add_node("check_order_info", check_order_info)
builder.add_node("ask_for_order_info", ask_for_order_info)
builder.add_node("handle_missing_order_info", handle_missing_order_info)
builder.add_node("judge_refund_amount", judge_refund_amount)
builder.add_node("auto_refund", auto_refund)
builder.add_node("human_review", human_review)
builder.add_node("handle_human_result", handle_human_result)

# 添加起点
builder.add_edge(START, "classify_intent")

# 条件分支 1：根据意图分流
builder.add_conditional_edges(
    "classify_intent",
    route_by_intent,
    {
        "refund": "check_order_info",
        "general": "handle_general_question"
    }
)

# 普通问题直接结束
builder.add_edge("handle_general_question", END)

# 条件分支 2：检查订单信息是否完整
builder.add_conditional_edges(
    "check_order_info",
    route_by_order_info,
    {
```

```
        "complete": "judge_refund_amount",
        "incomplete": "ask_for_order_info",
        "failed": "handle_missing_order_info"
    }
)

# 循环重试：用户补充信息后，再次检查订单信息
builder.add_edge("ask_for_order_info", "check_order_info")

# 信息仍不完整，结束流程
builder.add_edge("handle_missing_order_info", END)

# 条件分支 3：判断是否需要人工审核
builder.add_conditional_edges(
    "judge_refund_amount",
    route_by_refund_amount,
    {
        "auto": "auto_refund",
        "human": "human_review"
    }
)

# 自动退款结束
builder.add_edge("auto_refund", END)

# 人工审核后，处理审核结果
builder.add_edge("human_review", "handle_human_result")
builder.add_edge("handle_human_result", END)

# 编译图
graph = builder.compile()


if __name__ == "__main__":
    result = graph.invoke({
        "user_input": "我想申请退款，但是现在还没写订单号",
        "intent": "",
        "order_id": "",
        "refund_amount": 0.0,
        "retry_count": 0,
        "max_retry": 2,
        "need_human_review": False,
        "human_approved": False,
        "answer": ""
    })

    print("\n最终回复：")
    print(result["answer"])
```

3.4.4 运行效果一：普通问题分支

如果初始输入改成：

```
"user_input": "请问你们的营业时间是什么？"
```

流程会走：

```
START
 ↓
classify_intent
 ↓
handle_general_question
 ↓
END
```

输出类似：

```
最终回复：
您好，您的问题已收到。这是一个普通客服问题，我会为您提供常规帮助。
```

这个过程体现了 **条件分支模式。**

Agent 先判断用户意图，如果不是退款问题，就不会进入复杂退款流程。

3.4.5 运行效果二：订单信息不足时循环追问

如果初始输入是：

```
"user_input": "我想申请退款，但是现在还没写订单号"
```

系统会先进入退款流程，但检查订单信息时发现没有订单号，于是触发追问：

```
Agent：为了继续处理退款，请提供订单号。
用户补充：
```

此时输入：

```
我的订单号是 ORD-20260724-001
```

流程会回到：

```
ask_for_order_info
 ↓
check_order_info
```

然后继续执行后续退款判断。

这体现了 **循环重试模式**：

```
检查订单信息
  ├── 信息完整 → 继续
  └── 信息不足 → 追问用户 → 再次检查
```

通过 `retry_count` 和 `max_retry`，我们可以避免无限循环。

3.4.6 运行效果三：小额退款自动处理

如果用户补充的是普通订单号：

```
我的订单号是 ORD-20260724-001
```

系统会模拟退款金额为：

```
refund_amount = 99.0
```

流程会走：

```
judge_refund_amount
 ↓
auto_refund
 ↓
END
```

输出类似：

```
最终回复：
订单 ORD-20260724-001 已通过自动退款规则，退款金额 99.0 元，预计 1-3 个工作日原路返回。
```

这体现了通过条件分支实现自动化处理。

3.4.7 运行效果四：大额退款进入人工审核

如果用户输入中包含“大额”或“5000”，例如：

```
"user_input": "我想申请大额退款，订单号是 ORD-20260724-001，金额 5000"
```

系统会模拟退款金额为：

```
refund_amount = 5000.0
```

于是流程进入人工审核：

```
系统提示：该退款申请需要人工审核。
订单号：ORD-20260724-001
退款金额：5000.0
人工审核是否通过？请输入 yes / no：
```

如果输入：

```
yes
```

输出：

```
最终回复：
订单 ORD-20260724-001 的大额退款申请已通过人工审核，退款金额 5000.0 元，后续将进入退款流程。
```

如果输入：

```
no
```

输出：

```
最终回复：
订单 ORD-20260724-001 的大额退款申请未通过人工审核，建议联系人工客服了解具体原因。
```

这体现了 **Human-in-the-loop 模式。**

在真实生产环境中，这里的 `input()` 不会出现在服务端代码里，而是会替换成：

- 审批系统
- 工单系统
- 企业微信 / 飞书审批
- 管理后台
- LangGraph interrupt / checkpoint 机制

3.4.8 案例中的 LangGraph 结构分析

这个案例虽然业务规则很简单，但已经包含了 LangGraph 的核心能力。

能力	对应代码	作用
State 状态管理	`CustomerServiceState`	保存用户输入、意图、订单号、退款金额、重试次数等
Node 节点	`classify_intent`、`check_order_info` 等	每个函数负责一个处理步骤
条件分支	`add_conditional_edges`	根据意图、订单状态、金额选择不同路径

能力	对应代码	作用
循环重试	`ask_for_order_info → check_order_info`	信息不足时重新进入检查节点
Human-in-the-loop	`human_review`	大额退款进入人工审核
Agent 节点	`intent_agent = create_agent(...)`	使用 Agent 完成意图识别
LLM	`ChatOpenAI`	为 Agent 提供语言理解能力

从流程角度看：

```
LangGraph 负责编排
create_agent 负责智能判断
ChatOpenAI 负责语言理解
普通 Python 函数负责业务规则
```

这正是生产级 Agent 系统的常见分层方式。

3.4.9 为什么这个案例比普通 Workflow 更适合用 LangGraph

如果用普通 Workflow 写这个案例，也不是不行。
但代码很快会变成大量嵌套的 `if/else` 和 `while`：

```python
if intent == "refund":
    while order_id is None:
        ask_user()
    if refund_amount > 1000:
        human_review()
    else:
        auto_refund()
else:
    general_answer()
```

当业务继续扩展时，例如：

- 加入物流问题
- 加入投诉升级
- 加入黑名单判断
- 加入订单状态查询
- 加入退款失败重试
- 加入用户满意度回访
- 加入人工客服转接

普通 Workflow 会越来越难维护。

而 LangGraph 把流程拆成了清晰的节点和边：

```
一个节点负责一件事
一个条件边负责一次路由
一个 State 负责保存上下文
一个 Graph 负责整体流程
```

所以 LangGraph 的优势不是“代码更短”，而是：

- 流程更清楚
- 节点更容易复用
- 分支更容易扩展
- 状态更容易追踪
- 更适合加入人工审核和中断恢复
- 更适合生产环境监控和调试

教学小结：

LangGraph 最适合处理“有状态、有分支、有循环、有人工介入”的 Agent 任务。
本案例中，客服退款流程同时使用了条件分支、循环重试和 Human-in-the-loop，展示了 LangGraph 从普通 Workflow 升级而来的核心价值。